

JAVASCRIPT ESSENTIALS

HISTORY

1995

Brendan Eich

Netscape

10 days

BROWSER WAR

VS





STANDARDIZATION

- ECMA INTERNATIONAL IS AN INDUSTRY ASSOCIATION DEDICATED TO THE STANDARDIZATION OF INFORMATION AND COMMUNICATION SYSTEMS
 - ECMAScript
 - OFFICE OPEN XML

ECMAScript

- ES3 (1999) — “CLASIC” JAVAScript
- ES4 (CANCELADO) — THE GREAT FAILURE
- ES5 (2009) — JAVAScript IS STARTING TO MATURE
- ES6 / ES2015 — THE VERSION THAT CHANGES EVERYTHING
- ES2016/ES2015 — CONTINUOUS AND STABLE EVOLUTION



HOW IS
JAVASCRIPT
CURRENTLY
USED?

Frontend

Backend

Mobile applications

Desktop

IA

JAVASCRIPT ENGINES

- JAVASCRIPT ENGINES ARE INTERPRETERS THAT PARSE AND EXECUTE JAVASCRIPT CODE. MODERN JAVASCRIPT ENGINES USE JUST-IN-TIME (JIT) COMPILATION TO CONVERT JAVASCRIPT CODE INTO MACHINE CODE THAT CAN BE EXECUTED BY A COMPUTER'S PROCESSOR

JAVASCRIPT ENGINES

V8

SpiderMonkey

JavaScriptCore

Ryan Dalh 2009

```
graph TD; A[Ryan Dalh 2009] --> B[Use V8 engine]; B --> C[Event-Driven + Non-Blocking I/O];
```

Use V8 engine

Event-Driven + Non-Blocking I/O

NODE.JS

NODE.JS

Realtime

Streaming

Chats

Websockets

Development tools

INSTALL AND TEST JAVASCRIPT WITH NODE.JS

<https://nodejs.org/en/download>

Check

- `node -v`
- `npm -v`

Create a .js file

- `console.log("Hello word")`

Run the file

- `node app.js`

VARIABLES



var



let



const

DATA TYPES

string

number

bigint

boolean

undefined

symbol

null

TYPE OF OPERATOR

typeof value

Return a string with
the data type

VARIABLE SCOPE

- A VARIABLE MAY BELONG TO ONE OF THE FOLLOWING SCOPES:
 - GLOBAL SCOPE: THE DEFAULT SCOPE FOR ALL CODE RUNNING IN SCRIPT MODE.
 - MODULE SCOPE: THE SCOPE FOR CODE RUNNING IN MODULE MODE.
 - FUNCTION SCOPE: THE SCOPE CREATED WITH A FUNCTION.
- IN ADDITION, VARIABLES DECLARED WITH LET OR CONST CAN BELONG TO AN ADDITIONAL SCOPE:
 - BLOCK SCOPE: THE SCOPE CREATED WITH A PAIR OF CURLY BRACES (A BLOCK).

GLOBAL PROPERTIES

- PREDEFINED MEMBERS OF THE GLOBAL OBJECT THAT ARE AVAILABLE IN EVERY PART OF YOUR CODE
 - VALUE PROPERTIES
 - FUNCTION PROPERTIES

globalThis

Infinity

NaN

undefined

GLOBAL
VALUE
PROPERTIES

GLOBAL FUNCTION PROPERTIES

`eval()`

`parseInt()/parseFloat()`

`isNaN()`

`isFinite()`

`encodeURIComponent()/decodeUri()`

`escape()/unescape()`

ARITHMETIC OPERATORS

JAVASCRIPT SUPPORTS STANDARD
ARITHMETIC:

- +, ++
- -, --
- *
- /
- % (REMAINDER)
- ** (EXPONENTIATION)

COMPARISON OPERATORS

- **USE === (STRICT EQUALITY)**
- **!== (STRICT INEQUALITY)**
- **AVOID == AND != UNLESS YOU INTENTIONALLY WANT COERCION.**

LOGICAL OPERATORS

- **&& (AND), || (OR), AND ! (NOT)**

CONTROL FLOW

```
const score = 85;

// Determine the letter grade based on score range
if (score >= 90) {
  console.log('Grade: A');
} else if (score >= 80) {
  console.log('Grade: B'); // This branch executes
} else if (score >= 70) {
  console.log('Grade: C');
} else {
  console.log('Grade: F');
}
```

```
// switch for day-of-week handling
const day = 'Monday';

switch (day) {
  case 'Monday':
  case 'Tuesday':
  case 'Wednesday':
  case 'Thursday':
  case 'Friday':
    console.log(`${day} is a weekday`);
    break;
  default:
    console.log(`${day} is a weekend day`);
}
```

CONTROL FLOW

```
// for loop – iterate a known number of times
for (let i = 0; i < 5; i++) {
  console.log(`Iteration ${i}`);
}

// while loop – iterate until a condition is false
let count = 3;
while (count > 0) {
  console.log(`Countdown: ${count}`);
  count -= 1; // Decrement the counter
}
```

```
// for...of – iterate over array elements directly
const fruits = ['apple', 'banana', 'cherry'];
for (const fruit of fruits) {
  console.log(`Fruit: ${fruit}`);
}
```


EJERCICIO 01

- ESCRIBIR UN INTERPRETE PARA ANALIZAR TIPOS DE DATOS: NUMEROS (ENTEROS/DECIMALES), CADENAS, Y BOOLEANS
- SI SE TRATA DE UN NUMERO DEFINIR SI ES DECIMAL O ENTERO Y SU SIGNO (ANALISIS NUMERICO)
- SI ES UNA CADENA QUE PUEDE SER CONVERTIDA A NUMERO, REALIZAR EL ANALISIS NUMERICO, CASO CONTRARIO SOLO REPORTAR QUE ES UNA CADENA
- SI ES UN TIPO NO SOPORTADO MOSTRAR UN MENSAJE

EJEMPLO DE USO

```
console.log("-----");  
let valor = -15;  
console.log("Entrada:", valor);  
  
const resultado = procesarEntrada(valor);  
  
console.log("Resultado:", resultado);  
console.log("-----");
```


TYPE COERCION

- TYPE COERCION IS THE AUTOMATIC OR IMPLICIT CONVERSION OF A VALUE FROM ONE DATA TYPE TO ANOTHER BY THE PROGRAMMING LANGUAGE. THIS OCCURS DURING RUNTIME WHEN AN OPERATOR ENCOUNTERS UNEXPECTED DATA TYPES.

BOOLEAN COERCION

THIS HAPPENS AUTOMATICALLY IN LOGICAL CONTEXTS, SUCH AS IF STATEMENTS, LOOPS (WHILE), OR LOGICAL OPERATORS (&&, ||, !).

COERCED TO FALSE

FALSY VALUES: FALSE, 0, -0, 0n (BIGINT ZERO), "" (EMPTY STRING), NULL, UNDEFINED, AND NAN.

COERCED TO TRUE

TRUTHY VALUES: EVERYTHING ELSE, INCLUDING ARRAYS AND OBJECTS

EJERCICIO 02

- MEJORAR EL INTERPRETE ANTERIOR, INCLUYENDO SOPORTE PARA:
 - INIFINTY
 - BIGINT
 - NAN
- INCLUIR EN LA RESPUESTA SI EXISTIO COERCION

STRING MANIPULATION

TEMPLATE LITERALS

TEMPLATE LITERALS USE BACKTICKS AND ALLOW EMBEDDED EXPRESSIONS WITH `${EXPRESSION}`.
THEY ALSO SUPPORT MULTI-LINE STRINGS WITHOUT ESCAPE CHARACTERS

```
// Template literals use backticks for string creation  
const firstName = 'Jane';  
const lastName = 'Doe';  
  
// Embed expressions inside ${...}  
const fullName = `${firstName} ${lastName}`;  
console.log(fullName); // "Jane Doe"
```

STRING MANIPULATION

charAt(), toUpperCase(), toLowerCase(), includes(), startsWith(), endsWith(), slice(), substring(), trim(), split(), indexOf() and replace().

```
// Declare a sample string  
const text = ' Hello, JavaScript World! ';  
  
// Remove Leading and trailing whitespace  
const trimmed = text.trim();  
console.log(trimmed); // "Hello, JavaScript World!"
```


EJERCICIO 03

- MODIFICAR EL EJERCICIO DEL INTERPRETE DE TIPOS PARA AHORA EN VEZ DE RECIBIR UNA VARIABLE CON UNA CADENA LARGA QUE PUEDE TENER MULTIPLES VALORES.
- SEPARAR LA CADENA USANDO EL ESPACIO COMO CARACTER IDENTIFICADOR Y ANALIZAR CADA TOKEN PARA MOSTRAR SU TIPO Y POR ULTIMO MOSTRAR UN RESUMEN CON EL NUMERO DE TOKENS DE CADA TIPO

REGULAR EXPRESSIONS

- REGEX PATTERNS MATCH TEXT USING SPECIAL SYNTAX.
- `TEST()` TO CHECK FOR A MATCH
- `MATCH()` TO EXTRACT MATCHED SUBSTRINGS.

REFERENCES

- [HTTPS://WWW.WIRFS-BROCK.COM/ALLEN/JSHOPL.PDF](https://www.wirfs-brock.com/allen/jshopl.pdf)
- [HTTPS://ECMA-INTERNATIONAL.ORG/](https://ecma-international.org/)
- [HTTPS://WWW.TENSORFLOW.ORG/JS](https://www.tensorflow.org/js)
- [HTTPS://WWW.W3SCHOOLS.COM/Js/JS_es6.ASP](https://www.w3schools.com/Js/JS_es6.asp)
- [HTTPS://DEVELOPER.MOZILLA.ORG/EN-US/DOCS/GLOSSARY/ENGINE/JAVASCRIPT](https://developer.mozilla.org/en-US/docs/Glossary/Engine/JavaScript)
- [HTTPS://V8.DEV](https://v8.dev)
- [HTTPS://SPIDERMONKEY.DEV](https://spidermonkey.dev)
- [HTTPS://DEVELOPER.APPLE.COM/DOCUMENTATION/JAVASCRIPTCORE](https://developer.apple.com/documentation/javascriptcore)
- [HTTPS://NODEJS.ORG/EN](https://nodejs.org/en)
- [HTTPS://DEVELOPER.MOZILLA.ORG/EN-US/DOCS/WEB/JAVASCRIPT/GUIDE/](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/)
- [HTTPS://UNICODEPLUS.COM](https://unicodeplus.com)
- [HTTPS://REGEX101.COM](https://regex101.com)
- [HTTPS://REGEXR.COM](https://regexr.com)